

From Concept to Code

UTKRRANTI

www.utkrranti.com

pranit@utkrranti.com

+91-7768068585

26 June 2026

The Management,
Dr. Vithalrao Vikhe Patil Foundation
Medical College & Hospital, Ahilyanagar

**Subject: Proposal for Development of the VIMSMCH Vocational Training Website
(*training.vimsmch.edu.in*)**

Respected Sir / Madam,

I am pleased to submit this proposal for the design and development of a dedicated website for VIMSMCH's upcoming Vocational Training programmes. As the developer who has designed and maintained the main VIMSMCH website since inception, I have a thorough understanding of the institution's brand, operational requirements, and the expectations of its audience.

The vocational training initiative marks an important expansion of VIMSMCH's academic footprint — bringing structured, skill-based education to a wider community. A dedicated online presence is essential to communicate this effectively: to inform prospective students about available courses, enable them to express interest online, and provide a verifiable, permanent record of the certificates they earn.

This proposal presents a complete solution built on **Next.js** — a modern, production-grade web framework trusted by institutions globally — hosted on **Vercel's edge network** for guaranteed uptime and speed. The enclosed document covers every page that will be built, the full admin panel for your staff, a week-by-week delivery timeline, a transparent cost breakdown, and a comprehensive technical specification for your IT department.

I am confident this solution will serve VIMSMCH's vocational division professionally for years to come. I remain available to discuss any aspect of this proposal at your convenience.

Yours sincerely,

Utkrranti

Web Designer & Developer

pranit@utkrranti.com

+91-7768068585

Enclosed: Website Proposal & Technical Specification — June 2026

WEBSITE DEVELOPMENT PROPOSAL

From Concept to Code

UTKRRANTI

www.utkrranti.com

DR. VITHALRAO VIKHE PATIL FOUNDATION
Medical College & Hospital, Ahilyanagar

Vocational Training Website

Project Proposal

What we will build · How long it takes · What it costs

PREPARED FOR
VIMSMCH Management

PREPARED BY
Utkrranti

DATE
June 2026

What this document is: A proposal for a brand-new website that will live at **training.vismch.edu.in** and will serve as the complete online home for VIMSMCH's upcoming vocational training programmes — covering course listings, online applications, and student certificate verification.

1 The Idea in Simple Terms

VIMSMCH is launching vocational training courses — short-term, skill-based programmes for students who want practical, job-ready qualifications. Right now there is no dedicated online presence for these courses. Anyone who wants to know about a course, apply, or verify their certificate has no single place to go.

We propose building a clean, professional website on a subdomain that does three things:



Inform

Show all courses clearly with fees, duration, eligibility and syllabus



Accept Applications

Let students fill an online inquiry form — no paperwork to start



Verify Certificates

Employers and students can confirm a certificate is genuine online

Web address (subdomain): The new site will be at something like **training.vimsmch.edu.in** — completely separate from the main college website, with its own design, but carrying the same VIMSMCH brand and colours.

2 Every Page We Will Build

Below is every page that will be part of the new website. Think of each page as a room in a building — each one has a clear purpose for the person visiting.

#	Page	What it does (in plain English)
1	Home Page	The first thing every visitor sees. Shows a summary of all courses, key numbers (students trained, courses offered), and a big "Apply Now" button.
2	About the Vocational Centre	Explains what the vocational centre is, its goals, which bodies have approved it, and why VIMSMCH is the right choice.
3	Course Catalogue	A listing of all available courses — shown as cards with the course name, duration, fees, and a "Know More" button.
4	Individual Course Page	One dedicated page per course. Contains: what you will learn, who can apply, duration, fees, syllabus PDF, and the application button. (One template used for all courses.)
5	Admission & How to Apply	A step-by-step guide: eligibility → fill form → document list → fees → confirmation. Clear for first-time visitors.
6	Online Inquiry Form	A simple form: name, phone, email, course of interest, message. Submitted entries go to the admin panel and trigger an email alert to the office.
7	Faculty & Trainers	Photo cards for each trainer with their name, qualification, and subject. Builds trust for prospective students.
8	Facilities	Photos and descriptions of labs, equipment, classrooms, and infrastructure available to vocational students.
9	Placements & Outcomes	Where students went after completing courses — employer names, batch-wise pass rates, testimonials.
10	Gallery	Photos and videos from classes, events, and certificate ceremonies — updated through the admin panel.
11	Certificate Verification	Anyone (student, employer, college) can type in a certificate number and instantly see if it is genuine, the student name, course, and date issued.

#	Page	What it does (in plain English)
12	Contact Us	Office address, phone number, email, map, and working hours for the vocational centre.

Total: 12 pages. The Individual Course Page is built as one template that works for every course — so adding a new course later takes minutes, not days.

3 The Admin Panel — Managing the Website Yourself

Every piece of content on the website can be updated by your office staff — no coding needed. We build a private admin area (accessible only by authorised staff with a password) where they can manage everything listed below.

Admin Module	What the staff member can do
Courses	Add a new course, edit fees or duration, upload the syllabus PDF, set a course as active or hidden.
Inquiries	See all submitted inquiry forms in a table — name, phone, course interest, date. Mark them as contacted or resolved.
Batches	Create a new batch for a course, set start/end dates, add enrolled students.
Certificates	Add student certificate records (name, roll no, course, date) so they appear in the Certificate Verification page.
Gallery	Upload photos and videos for each course or event. Set a caption and order.
Announcements	Post notices (admission open, date changes, fee reminders) that appear on the home page.
Faculty	Add or update trainer profiles — name, photo, qualification.
Placements	Add testimonials and employer records shown on the Placements page.

No technical knowledge needed. The admin panel works in any web browser — your staff logs in with a username and password, and everything is point-and-click.

4 How It Will Be Built

This website will be built using **Next.js** — a modern web technology used by companies like Netflix, TikTok, and Notion. It is faster, more scalable, and easier to maintain than traditional approaches. The site will be hosted on **Vercel** (the platform built specifically for Next.js) and connected to a cloud database — meaning it can handle traffic spikes without any server management on your side.

Web Address	training.vimsmch.edu.in	A subdomain under your existing domain. DNS is pointed from Hostinger to Vercel — a 2-minute configuration change.
SSL / HTTPS	Vercel (auto-managed)	Vercel provisions and auto-renews an SSL certificate. The site is always secure with zero manual effort.
Framework	Next.js 14 (App Router)	The website and its admin panel are a single Next.js application — pages, API, and admin all in one codebase.
Language	TypeScript	Type-safe JavaScript. Catches errors at development time before they ever reach users.
Styling	Tailwind CSS	Utility-first CSS framework. Consistent, responsive design out of the box.
Database	PostgreSQL via Supabase	Free-tier cloud PostgreSQL database. Supabase provides a dashboard to view data directly, useful for IT audits.
ORM	Prisma	Type-safe database access layer. Schema is version-controlled — every DB change is tracked in Git.

Hosting	Vercel (Hobby / Pro)	Hobby plan is free for low traffic. Pro plan is \$20/month if needed — far cheaper than a VPS. No server to manage.
Auto-Deploy	Git push → Vercel	Every push to the main branch deploys automatically in ~60 seconds. No SSH, no rsync, no manual steps.
File Uploads	Vercel Blob / Cloudinary	Uploaded PDFs and images stored in cloud object storage — not on the server filesystem, so deployments never wipe uploads.

Why Next.js instead of PHP? The vocational website needs to be fast, SEO-friendly (so courses appear in Google search), and easy to scale as more courses are added. Next.js renders pages server-side for Google and client-side for users — giving the best of both worlds. It also separates this project cleanly from the existing college website with no shared risk.

5 Delivery Timeline — 6 Weeks

The project runs over six weeks. Each week has a clear set of deliverables — you will be able to review the work as it progresses, not just at the end.

Week 1 Foundation	▶ Subdomain + SSL setup on Hostinger ▶ Database structure created ▶ Admin panel skeleton (login, dashboard) ▶ Home Page design and development
Week 2 Core Pages	▶ About the Vocational Centre ▶ Course Catalogue page ▶ Individual Course page template ▶ Courses admin module (add / edit / delete courses)
Week 3 Applications & People	▶ Admission & How to Apply page ▶ Online Inquiry Form + email notification to office ▶ Inquiries admin module ▶ Faculty & Trainers page + admin
Week 4 Supporting Pages	▶ Facilities page ▶ Placements & Outcomes page + admin ▶ Gallery page + admin (photo upload) ▶ Announcements module
Week 5 Verification & Contact	▶ Certificate Verification page + admin (add certificate records) ▶ Contact Us page ▶ Batches admin module ▶ Mobile responsiveness check across all pages
Week 6 Testing & Launch	▶ Full cross-browser testing ▶ Content entry (initial courses, faculty etc.) ▶ Staff walkthrough of admin panel ▶ Go live + DNS confirmation

Review checkpoints: At the end of Week 2 and Week 4 you will receive a preview link to review the work done so far and request any adjustments before we proceed.

6 Investment — What It Costs

The total project is broken into two clear parts: the website pages the public sees, and the admin panel your staff uses to manage it.

Part A — Website Pages (12 pages)

#	Page	Rate
1	Home Page	₹ 2,500
2	About the Vocational Centre	₹ 2,500
3	Course Catalogue	₹ 2,500
4	Individual Course Page (template)	₹ 2,500
5	Admission & How to Apply	₹ 2,500
6	Online Inquiry Form	₹ 2,500
7	Faculty & Trainers	₹ 2,500
8	Facilities	₹ 2,500

#	Page	Rate
9	Placements & Outcomes	₹ 2,500
10	Gallery	₹ 2,500
11	Certificate Verification	₹ 2,500
12	Contact Us	₹ 2,500
Subtotal — 12 Pages		₹ 30,000

Part B — Admin Panel (8 modules)

Admin Module	Cost
Courses management (add / edit / delete courses + syllabus PDF)	₹ 5,000
Inquiries management (view, filter, mark as contacted)	₹ 3,000
Batches management (create batches, assign students)	₹ 3,000
Certificates management (add records for verification)	₹ 4,000
Gallery management (photo/video upload with captions)	₹ 2,000
Announcements module	₹ 1,500
Faculty management	₹ 1,500
Placements management (testimonials + employer records)	₹ 2,000
Subtotal — Admin Panel	₹ 22,000

Part C — Setup & Infrastructure

Subdomain creation + DNS configuration on Hostinger	₹ 1,000
Separate database setup and initial configuration	₹ 1,000
Deployment pipeline setup (GitHub Actions)	₹ 1,000
Subtotal — Setup	₹ 3,000

TOTAL PROJECT COST

₹ 55,000

12 pages (₹30,000) + Admin Panel (₹22,000) + Infrastructure Setup (₹3,000)

7 Payment Schedule

Payment is split into three milestones — you only pay for what has been delivered and approved.

Milestone 1
On project start — before work begins

₹ 27,500
50% advance

Milestone 2
After final deployment & go-live

₹ 27,500
50% on delivery

8 What's Included & What's Not

✓ Included in this proposal

- ▶ All 12 website pages designed and developed
- ▶ Full admin panel with 8 management modules
- ▶ Mobile-friendly design (works on phones and tablets)
- ▶ Online inquiry form with email notification
- ▶ Certificate verification system
- ▶ Subdomain and database setup
- ▶ Initial content entry (courses, faculty, first batch)
- ▶ Staff walkthrough training on admin panel
- ▶ 1 month of bug fixes after go-live

✗ Not included (separate scope)

- ▶ Online fee payment / payment gateway integration
- ▶ Student login portal or LMS (learning management system)
- ▶ Video hosting (YouTube / Vimeo links are fine, hosting is not)
- ▶ Content writing (text for courses, about page etc.)
- ▶ Logo design or branding work
- ▶ Annual Maintenance Contract (quoted separately if needed)
- ▶ WhatsApp / SMS notifications (available as add-on)

9 Next Steps

Step 1 Review this proposal

Go through this document. Note any changes — pages you want added, removed, or modified.

Step 2 Confirm course list

Share the list of vocational courses being launched so we can structure the Course Catalogue correctly from day one.

Step 3 Sign-off & advance

Once both parties agree on scope, share the 50% advance payment and we begin within 3 business days.

Step 4 Content inputs

Provide text, photos, and documents for the initial courses, faculty profiles, and about section as we go — we will send a checklist.

Step 5 Go live in 6 weeks

We handle everything technical. Your team only needs to approve the work at Week 2 and Week 4 checkpoints.

TECHNICAL ANNEX — FOR IT DEPARTMENT USE



Architecture & Implementation Specification

Next.js 14 · TypeScript · Prisma ORM · PostgreSQL · Vercel Edge Network
training.vimsmch.edu.in

This annex provides the complete technical specification for the VIMSMCH IT department. It documents the rendering architecture, data persistence layer, CI/CD pipeline, security posture, and API contracts for the vocational training subdomain — built as a production-grade Next.js 14 App Router application deployed on Vercel's global edge network with a Supabase-managed PostgreSQL backend.

T1

System Architecture & Runtime Model

Rendering Strategy — SSR / SSG / ISR per Route

Next.js 14 App Router uses React Server Components (RSC) as the default rendering primitive. Each route in the application is individually classified — the correct rendering strategy is chosen per page based on data volatility, personalisation requirements, and cache-ability:

Route	Strategy	Rationale
/courses, /about, /faculty, /facilities	ISR (revalidate: 3600)	Content changes infrequently. Pages are statically generated at build time and revalidated on a 1-hour TTL via stale-while-revalidate semantics. Zero DB queries on cache hit.
/courses/[slug]	ISR (revalidate: 3600)	generateStaticParams() pre-renders all course slugs at build time. New courses trigger on-demand revalidation via revalidatePath() Server Action.
/verify	SSR (no-store)	Certificate lookup is personalised per query param and must never be cached — rendered on the server per request with cache: "no-store" fetch directive.
/contact, /admission	Static (SSG)	Pure content pages with no dynamic data. Emitted as static HTML at build — served directly from Vercel CDN edge nodes, zero server invocation.
/admin/**	SSR (authenticated)	All admin routes are dynamically server-rendered. Next.js middleware intercepts unauthenticated requests before the RSC render cycle begins.
/api/inquiry (Route Handler)	Edge Runtime	Deployed to Vercel Edge Runtime for sub-10ms cold starts globally. Stateless — reads body, validates schema, writes to DB via connection pool, returns JSON.

Infrastructure Layer

Layer	Technology	Configuration
DNS	Hostinger DNS Manager	CNAME: training.vimsmch.edu.in → cname.vercel-dns.com. TTL 300s during cutover, 3600s post-stabilisation. No A record required.

Layer	Technology	Configuration
TLS/SSL	Vercel-managed Let's Encrypt	Vercel auto-provisions a TLS certificate for training.vimsmch.edu.in on domain add. Auto-renewal 30 days before expiry. Zero operational overhead.
CDN / Edge	Vercel Edge Network (100+ PoPs)	Static assets and ISR responses served from the nearest of 100+ global PoPs. Average TTFB for India: ~12ms from Mumbai edge.
Compute	Vercel Serverless Functions	Next.js Route Handlers compiled to isolated Node.js 20 serverless functions. Max execution: 10s (Hobby) / 60s (Pro). Cold start: ~200ms.
Edge Runtime	V8 Isolates on Vercel Edge	Inquiry submit Route Handler deployed to V8 isolate runtime — no Node.js bootstrap, ~5ms cold start, global distribution, 128MB memory limit.
Database	Supabase PostgreSQL 15	Managed PostgreSQL on AWS ap-south-1 (Mumbai). Connection pooling via PgBouncer in transaction mode (max_client_conn: 100, pool_size: 20). Direct URL for Prisma migrations; pooler URL for runtime.
File Storage	Vercel Blob (S3-backed)	Uploaded PDFs and images stored in Vercel Blob. URLs are stable, CDN-cached. Max file size: 500MB. Access controlled via signed tokens for admin uploads.
Email	Resend API / SMTP Relay	Transactional email (inquiry notifications) sent via Resend SDK or existing Hostinger SMTP. API key stored as Vercel environment variable — never in source.

T2 Data Model — Prisma Schema

The data layer is defined in a single `prisma/schema.prisma` file — the authoritative, version-controlled source of truth for the database schema. Migrations are generated via `prisma migrate dev` and applied atomically. All timestamps are stored as UTC. Soft deletes are implemented via a `deletedAt DateTime? nullable` field (`null` = active, `set` = deleted) — no row is ever permanently removed in production.

```
// prisma/schema.prisma (excerpt — key models)

model Course {
  id Int @id @default(autoincrement())
  title String @db.VarChar(200)
  slug String @unique @db.VarChar(220)
  description String @db.Text
  durationWeeks Int
  fees Decimal @db.Decimal(10,2)
  eligibility String @db.VarChar(500)
  syllabusPdf String? // Vercel Blob URL
  isActive Boolean @default(true)
  displayOrder Int @default(0)
  batches Batch[]
  inquiries Inquiry[]
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  deletedAt DateTime? // null = active
}

model Certificate {
  id Int @id @default(autoincrement())
  certificateNo String @unique @db.VarChar(50) // indexed, UNIQUE constraint
  studentName String @db.VarChar(200)
  course Course @relation(fields: [courseId], references: [id], onDelete: Restrict)
  batch Batch @relation(fields: [batchId], references: [id], onDelete: Restrict)
  issueDate DateTime @db.Date
  isValid Boolean @default(true)
  createdAt DateTime @default(now())
}

model Inquiry {
  id Int @id @default(autoincrement())
  fullName String @db.VarChar(100)
  phone String @db.VarChar(15)
  email String @db.VarChar(254)
  course Course? @relation(fields: [courseId], references: [id])
  message String? @db.Text
  status InquiryStatus @default(NEW)
  ipAddress String? @db.VarChar(45) // IPv4 + IPv6
  createdAt DateTime @default(now())
}

enum InquiryStatus { NEW CONTACTED CLOSED }
```

Referential integrity: All foreign key relations use `onDelete: Restrict` — a Course with associated Certificates or Batches cannot be deleted, only soft-deleted via `deletedAt`. The `certificateNo` field carries a PostgreSQL UNIQUE constraint enforced at the DB level, independent of application logic.

T3 Application Directory Structure — Next.js App Router

```

// Repository root
vismch-vocational/
├─ app/ ← Next.js App Router root
│  ├─ layout.tsx ← Root layout: global fonts, metadata, analytics
│  ├─ globals.css ← Tailwind base + CSS custom properties
│  ├─ (public)/ ← Route group — no auth, no layout wrapper cost
│  │  ├─ page.tsx ← / Home (ISR)
│  │  ├─ courses/page.tsx ← /courses (ISR)
│  │  ├─ courses/[slug]/page.tsx ← /courses/:slug (ISR + generateStaticParams)
│  │  ├─ verify/page.tsx ← /verify (SSR, no-cache)
│  │  ├─ about/page.tsx
│  │  ├─ admission/page.tsx
│  │  ├─ faculty/page.tsx
│  │  ├─ facilities/page.tsx
│  │  ├─ placements/page.tsx
│  │  ├─ gallery/page.tsx
│  │  └─ contact/page.tsx
│  ├─ (admin)/ ← Route group — protected by middleware auth guard
│  │  ├─ layout.tsx ← Admin shell: sidebar, nav, session context
│  │  └─ admin/
│  │     ├─ page.tsx ← Dashboard: inquiry count, recent activity
│  │     ├─ courses/page.tsx ← DataTable + Server Actions for CRUD
│  │     ├─ batches/page.tsx
│  │     ├─ inquiries/page.tsx
│  │     ├─ certificates/page.tsx
│  │     ├─ faculty/page.tsx
│  │     ├─ gallery/page.tsx
│  │     ├─ placements/page.tsx
│  │     └─ announcements/page.tsx
│  └─ api/ ← Route Handlers (replaces pages/api)
│     ├─ inquiry/route.ts ← POST — Edge Runtime, Zod validation, rate limit
│     ├─ verify/route.ts ← GET — no-cache, public
│     └─ auth/[...nextauth]/route.ts ← NextAuth.js v5 catch-all handler
├─ components/
│  ├─ ui/ ← Headless primitives (shadcn/ui)
│  ├─ CourseCard.tsx
│  ├─ InquiryForm.tsx ← Client Component — react-hook-form + Zod
│  └─ CertVerifyWidget.tsx ← Client Component — async fetch to /api/verify
├─ lib/
│  ├─ db.ts ← Prisma client singleton (global cached instance)
│  ├─ auth.ts ← NextAuth.js v5 config: Credentials provider, JWT
│  ├─ validators.ts ← Zod schemas shared between client + server
│  └─ ratelimit.ts ← Upstash Redis sliding-window rate limiter
├─ prisma/
│  └─ schema.prisma
├─ migrations/ ← Versioned migration history
├─ middleware.ts ← Edge Middleware: JWT validation, /admin/** guard
├─ next.config.js ← CSP headers, image domains, bundle analyser
├─ tailwind.config.ts
└─ package.json

```

T4

CI/CD Pipeline — Git → Vercel

Every commit triggers an automated pipeline. The developer pushes code — Vercel handles the build, test, and atomic deployment with zero-downtime promotion. Preview deployments are created for every pull request, giving a live URL per feature branch before it reaches production.

Stage	Trigger	Action & Outcome
1 · Type Check	git push (any branch)	tsc --noEmit — TypeScript compiler runs in strict mode. Any type error fails the pipeline immediately. No untyped code reaches Vercel.
2 · Lint	git push (any branch)	eslint with @typescript-eslint/recommended + next/core-web-vitals ruleset. Catches common RSC anti-patterns (async client components, missing Suspense boundaries).
3 · Build	push to main / PR	next build — emits optimised production bundle. Route manifests, RSC payloads, and ISR revalidation metadata written to .next/. Build fails on any Next.js compilation error.
4 · Preview Deploy	pull request opened	Vercel creates an immutable preview deployment at a unique URL (e.g. vocational-git-feat-xyz.vercel.app). Shareable for QA review before merge. Environment uses preview DB (separate Supabase branch).
5 · Production Deploy	merge to main	Vercel promotes the pre-built artifact (no rebuild) to production via atomic alias swap. Zero-downtime — old deployment serves traffic until new deployment passes health check.
6 · Post-Deploy Smoke	after production promote	Vercel deployment hook triggers a curl to /api/verify?cert=HEALTH_CHECK — expects HTTP 200 + {"valid":false}. Alerts on non-200 via email to pranit@utkrranti.com.
7 · Prisma Migrate	manual (pre-deploy hook)	prisma migrate deploy runs as a Vercel build command for schema-changing releases. Migrations are idempotent and tracked in prisma/migrations/.

Secret Management: All credentials (DATABASE_URL, DIRECT_URL, NEXTAUTH_SECRET, BLOB_READ_WRITE_TOKEN, SMTP API key) are stored as encrypted Vercel Environment Variables — scoped separately to Production, Preview, and Development environments. No secret ever touches the repository. Vercel injects them at build time and runtime via process.env. Rotation requires updating the Vercel dashboard only.

T5

Security Architecture

The application is written ground-up against the OWASP Top 10 (2021). Each control is implemented at the correct architectural layer — not bolted on as an afterthought.

Authentication — NextAuth.js v5	Credentials provider with bcrypt password verification (cost factor 12). On successful auth, NextAuth issues a signed JWT (HMAC-SHA256, secret = NEXTAUTH_SECRET env var, 8h expiry) stored in an HttpOnly, SameSite=Strict, Secure cookie. No session state on the server — fully stateless.
Route Protection — Edge Middleware	middleware.ts executes on the Vercel Edge Runtime for every request matching /admin/**. It decodes and verifies the JWT using jose (Web Crypto API) before the request reaches any RSC or Route Handler. Unauthenticated requests receive an immediate 307 redirect to /login — no DB query, no RSC execution.
CSRF Mitigation	Server Actions (used for all admin CRUD mutations) are protected by Next.js's built-in CSRF defence: the runtime validates that the request Origin header matches the deployment origin. Additionally, SameSite=Strict on the auth cookie prevents cross-origin requests from carrying credentials.
Input Validation — Zod	All external inputs (inquiry form, admin form submissions, API query params) are parsed through Zod schemas defined in lib/validators.ts. Schemas are shared isomorphically between client-side react-hook-form (for UX feedback) and server-side Route Handlers/Server Actions (for enforcement). A Zod parse failure returns HTTP 422 with a structured error map — no unvalidated data ever reaches Prisma.

SQL Injection — Prisma ORM	Prisma generates parameterised SQL for all queries via its query engine. Raw SQL is not used anywhere in the application. Even Prisma's <code>\$queryRaw</code> escape hatch is banned via ESLint rule (no-restricted-syntax). The ORM layer constitutes a complete SQLi firewall.
Rate Limiting — Upstash Redis	The <code>/api/inquiry</code> Route Handler enforces a sliding-window rate limit of 5 requests per 10 minutes per IP (extracted from <code>x-forwarded-for</code> , with spoofing protection). Implemented via <code>@upstash/ratelimit</code> with the Upstash Redis REST API. Exceeding the limit returns HTTP 429 with <code>Retry-After</code> header. No state is stored in the serverless function — Redis is the authoritative rate-limit counter.
File Upload Security — Vercel Blob	Uploads go directly to Vercel Blob via a presigned PUT URL. The Route Handler that issues the URL validates: (1) authenticated session, (2) MIME type from <code>Content-Type</code> header against whitelist [<code>application/pdf</code> , <code>image/jpeg</code> , <code>image/png</code> , <code>image/webp</code>], (3) file size $\leq$ 10MB. The blob store is not publicly writable — the presigned URL has a 60-second TTL.
HTTP Security Headers — <code>next.config.js</code>	Headers applied globally via <code>Next.js headers()</code> config: <code>Content-Security-Policy</code> (default-src 'self'; script-src 'self' + nonce for inline; img-src * data: blob:; frame-ancestors 'none'), <code>X-Frame-Options: DENY</code> , <code>X-Content-Type-Options: nosniff</code> , <code>Referrer-Policy: strict-origin-when-cross-origin</code> , <code>Permissions-Policy: camera=(), microphone=(), geolocation=()</code> . HSTS (max-age=63072000; includeSubDomains; preload) added by Vercel automatically for production deployments.
Dependency Hygiene	<code>package.json</code> uses exact versions for security-critical packages (next, next-auth, prisma). Dependabot configured to file PRs for patch + minor security updates weekly. npm audit runs in the CI pipeline — critical/high severity findings fail the build.

T6 Route Handler API Contracts

Two public-facing Route Handlers are exposed. Both implement the Response Web API (not Node.js `http.ServerResponse`) and run on the Edge Runtime. Request/response shapes are typed via Zod and inferred TypeScript interfaces.

POST /api/inquiry — Submit Inquiry Form

```
// Runtime: Edge | Auth: none | Rate limit: 5 req / 10 min / IP
// Request body (application/json)
interface InquiryPayload {
  fullName: string; // max 100 chars, trimmed
  phone: string; // /^[6-9]\d{9}$/ — Indian mobile number
  email: string; // RFC 5321 validated via Zod .email()
  courseId?: number; // optional — must reference existing course.id
  message?: string; // max 1000 chars
}

// 200 OK
{ "success": true }

// 422 Unprocessable Entity — Zod validation failure
{ "success": false, "errors": { "phone": ["Invalid Indian mobile number"] } }

// 429 Too Many Requests
{ "success": false, "error": "Rate limit exceeded" } + Retry-After: 480 header
```

GET /api/verify?cert={certificateNo} — Certificate Verification

```
// Runtime: Edge | Auth: none | Cache: no-store (Response header)
// Query param: cert — alphanumeric, max 50 chars (Zod validated)
// 200 OK — Certificate found and valid
{
  "valid": true,
  "certificateNo": "VOC-2026-001234",
  "studentName": "Priya Sharma",
  "course": "Phlebotomy Technician",
  "batch": "Batch 2 · Jan-Mar 2026",
  "issueDate": "2026-04-01"
}

// 200 OK — Not found (intentional 200 to prevent timing-based enumeration)
{ "valid": false }

// 400 Bad Request — cert param missing or format invalid
{ "error": "Invalid certificate number format" }
```

Server Actions — Admin CRUD

All admin create/update/delete operations are implemented as Next.js Server Actions ('use server' functions) rather than Route Handlers. Server Actions execute on the server, are automatically CSRF-protected by the Next.js runtime, and can call Prisma directly without an HTTP round-trip. They return typed `ActionResult<T>` discriminated unions consumed by client components via `useTransition` for non-blocking UI updates and optimistic state.

T7 Performance Engineering

Area	Implementation & Target Metric
Core Web Vitals Targets	$LCP \leq 2.5s$ (hero image served from Vercel CDN edge, preloaded via <code><link rel="preload"></code>). $CLS = 0$ (all images carry explicit width/height via <code>next/image</code>). $INP \leq 200ms$ (admin interactions use <code>useTransition</code> — React keeps UI responsive during Server Action round-trips).
next/image Optimisation	All <code></code> tags replaced with <code>next/image</code> . Vercel automatically serves WebP/AVIF via content negotiation, resizes to the requested display size, and lazy-loads below-fold images. Faculty and gallery images served from Vercel's image optimisation CDN with 1-year cache TTL.
Bundle Splitting & Tree Shaking	Next.js automatically code-splits at the route level. Shared vendor chunks (React, Tailwind runtime, shadcn components) are cached independently of page-specific JS. <code>next/dynamic</code> used for heavy admin-only components (rich text editor, data tables) to exclude them from public bundle. Target initial JS payload: $< 80\text{ KB}$ gzipped.
ISR Revalidation Strategy	Course and faculty pages use <code>revalidate: 3600</code> (1-hour ISR). When an admin publishes a new course via Server Action, <code>revalidatePath('/courses')</code> is called inline — the next request to <code>/courses</code> triggers an on-demand regeneration without waiting for the TTL. Stale-while-revalidate means users always get a cached response.
Database Connection Pooling	Prisma is instantiated as a global singleton (<code>lib/db.ts</code>) to survive hot reloads in dev. In production, Supabase PgBouncer handles connection pooling in transaction mode (<code>pool_size: 20</code>) — the serverless functions never exhaust PostgreSQL's <code>max_connections</code> . <code>DIRECT_URL</code> (non-pooled) is used only for prisma migrate deploy.
PostgreSQL Indexes	Explicit <code>@index</code> directives in <code>schema.prisma</code> : <code>Course.slug</code> (B-tree, unique), <code>Certificate.certificateNo</code> (B-tree, unique), <code>Inquiry.createdAt</code> (B-tree DESC for dashboard sort), <code>Course.isActive</code> + <code>Course.displayOrder</code> (composite, for catalogue listing query). Prisma generates the migration DDL automatically.